

A Transformer Based Pipeline for Software Requirements Classification

Maheen Mashrur Hoque
Islamic University of Technology
Gazipur, Bangladesh
maheenmashrur@iut-dhaka.edu

Ahsan Habib
Islamic University of Technology
Gazipur, Bangladesh
ahsanhabib@iut-dhaka.edu

Arman Hossain Dipu
Islamic University of Technology
Gazipur, Bangladesh
armanhossain@iut-dhaka.edu

Ajwad Abrar
Islamic University of Technology
Gazipur, Bangladesh
ajwadabrar@iut-dhaka.edu

Abstract

The automation of the software development lifecycle (SDLC) is a key challenge in the field of software development. One of the main challenges in automation of the SDLC lies within the work of collecting, analyzing and establishing requirements, where the requirements collected from the stakeholders are often noisy, which can be difficult to organize. In order to facilitate the automation of requirement analysis, our study proposes a cost-effective approach - leveraging the lightweight DistilBERT and RoBERTa models with a method called “Ensemble Pooling” to filter out relevant requirements. Our experiments showcased an accuracy of 78% for the “Ensemble Pooling” method, a higher score than readily available sequence classifier versions of the aforementioned models.

CCS Concepts

• Software and its engineering → Requirements analysis.

Keywords

Software Requirements, Software Development Lifecycle (SDLC), Classification, Natural Language Processing (NLP), Transformers, Ensemble

ACM Reference Format:

Maheen Mashrur Hoque, Arman Hossain Dipu, Ahsan Habib, and Ajwad Abrar. 2018. A Transformer Based Pipeline for Software Requirements Classification. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 5 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Software requirements provide high-level descriptions of a system’s expected functionality and constraints, serving as the primary medium through which stakeholder intent is communicated to development teams [13]. To be effective, requirements should

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 ACM.

ACM ISBN 978-1-4503-XXXX-X/18/06

<https://doi.org/XXXXXXX.XXXXXXX>

maintain clarity, consistency, and completeness [20]. However, real-world Software Requirements Specification (SRS) documents often fall short of this expectation due to communication gaps, inconsistent documentation practices, and varied stakeholder perspectives [12]. These issues result in ambiguous or overly verbose specifications that contain substantial portions of irrelevant or non-actionable information, hereafter referred to as “noise.” Such noise complicates analytical processes such as classification, prioritization, and traceability, increasing manual effort for analysts and creating delays within the Software Development Life Cycle (SDLC).

Addressing this challenge requires the ability to automatically distinguish actionable requirements from noisy text. We model this as a Natural Language Processing (NLP) classification task, where a system must determine whether a given sentence constitutes a requirement. In addition to accuracy, cost efficiency is an essential design objective since many organizations and researchers operate with limited computational resources. Motivated by these considerations, we explore the use of lightweight transformer architectures as an effective and scalable solution for requirement noise reduction.

Building on the baseline established by Ivanov et al. [7], we propose an ensemble-based pooling pipeline that leverages contextual embeddings extracted from compact transformer models. The approach integrates these embeddings with ensemble classifiers to improve prediction performance while remaining computationally efficient. The key contributions of this paper are as follows:

- A cost-efficient transformer-based pipeline that uses DistilBERT and RoBERTa with ensemble pooling for requirement classification.
- A novel ensemble pooling strategy that combines transformer embeddings with robust tree-based and boosting-based classifiers.
- An empirical comparison against prior work using the RegExpURE dataset, demonstrating consistent improvements over BERT-based baselines and traditional shallow models.

2 Related Work

This section reviews prior studies relevant to requirement classification and noise reduction. To provide a broad perspective, we include both traditional rule-based techniques and transformer-based approaches.

2.1 Traditional and Rule-based Approaches

Siahaan and Darnoto introduced the MultiPhiLDA method for detecting irrelevant requirements by modeling topic and word distributions of actor and action terms using polynomial probability functions [16]. Alharbi et al. leveraged semantic embeddings to identify ambiguous requirements, where the embeddings were fine-tuned to capture context-specific indicators of ambiguity [1]. Malik et al. focused on detecting conflicting requirements with cosine similarity over sentence embeddings, which, although aimed at conflicts, provides insight into similarity-based classification techniques [10]. Norabid et al. proposed a rule-based linguistic method that defines extraction rules for dependency relations and part-of-speech patterns, enabling the construction of subject-relation-object triples for downstream analysis [11].

2.2 Transformer-based Approaches

Ivanov et al. investigated requirement extraction using the BERT model [4] fine-tuned on an annotated subset of the PURE corpus [6]. Their model achieved an accuracy of 74 percent, outperforming fastText, which obtained 60 percent accuracy, and ELMo with an SVM classifier, which achieved 59 percent [7]. Since their task and dataset align closely with the objective of distinguishing requirement sentences from non-requirement ones, their work serves as the primary baseline for our study.

3 Dataset

This study uses the PURE (Public Requirements) dataset [6]. It contains 79 Software Requirements Specification (SRS) documents with a total of 34,286 sentences. The dataset provides only raw text. It does not include labels for identifying requirement sentences. Several earlier works, such as [7] and [16], created their own labels to support NLP tasks.

For our experiments, we use the RegExpPURE subset introduced by Ivanov et al. [7]. This subset contains sentence-level labels and has a balanced distribution. The training set has 2474 non-requirements and 2832 requirements. The test set has 467 non-requirements and 1058 requirements. The validation set has 255 non-requirements and 650 requirements. We use the same splits as the authors to ensure a fair comparison with their baseline model.

4 Methodology

Based on the approaches discussed in section 2, we adopt transformer models for the requirement classification task. Our method focuses on a uni-modal setup that uses only text data. This choice is practical because requirement documents are typically provided in textual form, and most information needed for classification is already present in the written descriptions. A multimodal extension, similar to the direction explored by Norabid et al. [11], may be useful in future work, but it is outside the scope of the current study.

4.1 Selected Transformer Models

We use two lightweight transformer models for this study, DistilBERT and RoBERTa. Both models belong to the transformer family, which relies on the self-attention mechanism to model relationships between tokens in a sequence. This mechanism captures long-range

dependencies more reliably than recurrent architectures such as RNNs or LSTMs, which often struggle with vanishing gradients and sequential processing constraints [19]. Due to this capability, transformer models have become a standard choice for classification tasks involving natural language.

DistilBERT is a compact version of the original BERT model. It is produced through a student-teacher distillation process that reduces the number of layers while preserving much of the semantic knowledge present in the teacher network [14]. The model matches the teacher's behavior by approximating its logits, hidden states, and attention patterns during training. DistilBERT retains a substantial portion of BERT's accuracy on several NLP benchmarks while requiring significantly fewer parameters. Its reduced memory footprint and computational cost make it suitable for scenarios where efficiency is an important constraint. This motivates its use in our work, as our goal is to design a method that remains accessible without requiring high-end hardware.

RoBERTa is an optimized variant of BERT that builds upon the same architecture but introduces several improvements intended to strengthen the model's language representations [9]. It is trained on a larger and more diverse text corpus and uses dynamic masking during pre-training, which changes the masked positions across epochs. This exposes the model to a wider range of contexts and reduces overfitting to fixed masked patterns. RoBERTa also removes the Next Sentence Prediction task and adjusts key hyperparameters such as batch size and training duration. These changes collectively improve stability during training and boost downstream performance. RoBERTa consistently achieves stronger results than BERT on many NLP tasks, which makes it a strong yet still relatively efficient candidate for requirement classification. For these reasons, we include it alongside DistilBERT in our experiments.

4.2 Definition of Noise in Requirements

In the context of software requirements, noise can be referred to as irrelevant information or non-essential details that do not contribute to the actual functionality or goals of the system. These noises can be redundant information, ambiguous language or non-requirements. To illustrate an example from the RegExpPURE dataset [7], the following is a clear requirement:

System Initialization shall [SRS014] initiate the watchdog timer.

And the following is a noise:

A Working Group was established in May, 2006 to develop high-level functional specifications for an NLM Digital Repository for NLM collection materials and to identify policy and management issues related to the creation, design and management of the repository.

We can see that in order to recognize what is a requirement and what is not a requirement, one needs to read the entire sentence and map the terms of the sentences to clarify what the sentence expresses. This can be used as an analogy of what the term 'long range dependency' entails for transformer models, the model's awareness of the expressive meaning of every term in the sequence with one another.

4.3 Ensemble Pooling Approach

For this classification task of filtering noise, we propose a method that we call “Ensemble Pooling”. Our approach consists of three layers -

- (1) **Tokenizer Layer** : This layer will appropriately tokenize the input sequence from text to corresponding vector representation.
- (2) **Transformer Layer** : This layer will produce the contextual embedding for the input sequence by feeding into the vector representation of the tokenizer layer.
- (3) **Classifier Layer** : This layer will use the contextual embedding (via pooling) produced by the transformer layer to classify the requirements from the noise. This layer would be trained with appropriate labels for classification. In our experimentation, we got better result with ensemble classifiers (hence the name, ‘Ensemble Pooling’).

The following Figure 1 represents the architecture of our approach.

Here, we utilized the last hidden layer of the transformer models, which contains integrated information or the final contextualized embeddings from the entire sequence in a multidimensional vector representation (in the Huggingface Transformer library [14], the dimension is $(Seq_{size} \times Seq_{length} \times Size_{hidden})$ the default hidden size is 768). But the most important content of the layer is the [CLS] token representation, which is added to each of the embeddings vector by the tokenizer and the vector for that token acts as the aggregate representation of the entire sequence [4] [15], that we use with the ensemble classifiers for classification tasks. The DistilBertForSequenceClassification and RobertaForSequenceClassification (based on DistilBERT and RoBERTa) classifiers offered by the Transformers library uses a similar approach, but with linear classification layer on top. These were compared as baselines.

Note that the transformer layer does not have any training cycle. This is because the transformer models used here (BERT and RoBERTa) are already trained over a large corpus of the English language and already have the necessary vocabulary structuring for language related tasks [14] [9]. This facilitates in reducing the cost and time that one had to invest with training.

4.4 Classification Layer

The classification layers were trained over the pooled vectors from the transformer layer and the labels from the dataset. The labels were converted to 0 for not a requirement (i.e. noise) and 1 for requirement (i.e. a proper requirement). Then 4 classifier models were trained and evaluated, which were Random Forest Classifier, XGB Classifier, LightGBM classifier and SVM Classifier. Out of these, 3 are ensemble models (Random Forest, XGB, LightGBM). We chose ensemble models because they leverage multiple learning algorithms (or sub-classifiers) to obtain better predictive performance than could be obtained from any of the standalone models. Moreover, due to combining sub-classifier, ensemble models are robust against overfitting [2]. On the other hand, SVM is not an ensemble model, rather it is a standalone classifier that determines a hyperplane to best separate the datapoints into classes [3]. SVM was also experimented with as it had shown good performance in NLP related classification tasks in several works [5] [18] [8].

4.5 Training Parameters

For the experiments, the core DistilBERT and RoBERTa models, used for ensemble pooling were not trained specifically for this study. However the two transformer based classifiers and the other classifiers were trained. The transformer based classifier models were fine tuned for classification, with the hyperparameters being the same for both - Batch Size = 16, Max Length = 128, Learning Rate = 2×10^{-5} and was trained over 10 epochs. Adam optimizer was used and Cross Entropy Loss was the loss function for both of the models. Both DistilBERT and RoBERTa is based on the BERT model. Their parameters have no difference. Where they do differ is in how they handle sequences in their inner layers (which is discussed in section 4)

For the classifiers, all were fine tuned with 10 fold grid search cross validation. The target metric for the hyperparameter tuning was accuracy. The optimal hyperparameters for the models are as follows in Table 2, Table 1, Table 4, and Table 3.

Table 1: Random Forest Hyperparameters for RoBERTa and DistilBERT Pairings

Hyperparameter	RoBERTa	DistilBERT
Max Depth (max_depth)	None	10
Min Samples Leaf (min_samples_leaf)	4	4
Min Samples Split (min_samples_split)	2	2
Number of Estimators (n_estimators)	300	100

Again, the term “Pairing” is referred to as using the hidden layer of the language models through the classifier models to make predictions.

Table 2: LightGBM Hyperparameters for RoBERTa and DistilBERT Pairings

Hyperparameter	RoBERTa	DistilBERT
Colsample by Tree (colsample_bytree)	1.0	1.0
Learning Rate (learning_rate)	0.1	0.1
Min Child Samples (min_child_samples)	20	50
Number of Estimators (n_estimators)	300	200
Num Leaves (num_leaves)	50	50
Subsample (subsample)	0.8	0.8

Table 3: XGBoost Hyperparameters for RoBERTa and DistilBERT Pairings

Hyperparameter	RoBERTa	DistilBERT
Learning Rate (learning_rate)	0.1	0.1
Max Depth (max_depth)	5	5
Number of Estimators (n_estimators)	300	300
Subsample (subsample)	0.8	0.7
Colsample by Tree (colsample_bytree)	0.8	1.0
Gamma (gamma)	0.1	0

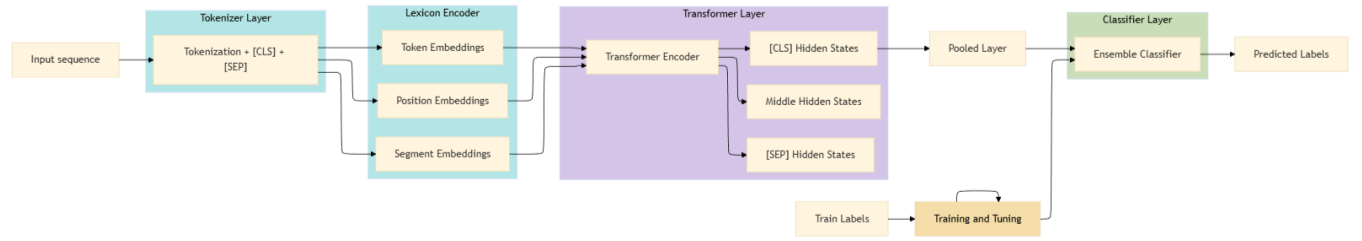


Figure 1: The Ensemble Pooling Classification Approach

Table 4: SVM Hyperparameters for RoBERTa and DistilBERT Pairings

Hyperparameter	RoBERTa	DistilBERT
C (C)	10	10
Gamma (gamma)	scale	auto
Kernel (kernel)	rbf	rbf

Table 5: Results Comparison of the Various Approaches

Approach Category	Model / Technique	Accuracy (%)
DistilBERT	DistilBERT Sequence Classifier	76
DistilBERT	DistilBERT + Random Forest	78
DistilBERT	DistilBERT + XGB	77
DistilBERT	DistilBERT + LightGBM	77
DistilBERT	DistilBERT + SVM	76
RoBERTa	RoBERTa Sequence Classifier	77
RoBERTa	RoBERTa + Random Forest	75
RoBERTa	RoBERTa + XGB	77
RoBERTa	RoBERTa + LightGBM	77
RoBERTa	RoBERTa + SVM	77

Table 6: Results of the Baseline Paper [7]

Approach	Accuracy (%)
fastText	60
ELMo + SVM	59
BERT	74

5 Results

As per the setup described in section 4, we compared our experiments with the baseline paper. Similar to the baseline work, results are based on the validation set. For the result comparison, primarily used the accuracy metric (The rationale behind this choice of metric is discussed in the section 6). The Table 5 contains the results and the Table 6 shows the baseline paper result.

For our experiments and proposed approach of identifying requirement noise, we used the RegExpPURE dataset [7]. So, our baseline was the results in that paper for the dataset in which they

utilized the BERT base model for extracting requirements. Though their work may not directly state noise reduction, but their work achieved similar objectives. As seen in section 3 their dataset consists labeled sentences of requirements and non requirements. And these non requirement sentences are precisely what we have defined as noise. So, the rationale for using this paper as a baseline is quite reasonable.

6 Discussion

6.1 The Rationale for Accuracy

Accuracy is intuitive and easy to understand. It directly reflects the overall correctness of the model's predictions. In many scenarios, balanced performance across different classes is desirable. Accuracy treats all classes equally. Moreover, when the class distribution is roughly uniform (which is the case for the RegExpPURE Dataset), accuracy is a reasonable choice. And hence, accuracy was the chosen metric. [17].

6.2 Base Classifier vs Ensemble Pooling

From the results in Table 5 and Table 6 shows that our approach combining DistilBERT and Random Forest in ensemble pooling performs better in terms of accuracy when compared against all the approaches and baseline results. However, all the approaches with RoBERTa had quite close results, except Random Forest. This deviation is most likely due to how RoBERTa utilizes byte-pair encoding in tokenization and how the inner encoder layers attune to the context of the sequences [9].

The baseline paper showed the most promising result with BERT with about 74% in accuracy. But baseline classifier of BERT, similar to DistilBERT and RoBERTa uses a neural network layer with a linear softmax activation function to generate class probability distribution. While effective in most cases, this classification task can benefit from more robust method of classification, ensemble classification. Since ensemble classifiers use bagging of sub classifiers, it is more robust in classification. Random Forest is one such ensemble method that ensembles decision trees. These trees or sub classifiers work in tandem for classification and improves accuracy, as seen from our experiments. Again, the rationale behind choosing DistilBERT and RoBERTa comes to their lightweight performance. They require much less computing resources compared to some other larger models and still remains a popular choice for smaller NLP related works.

However, the lightweight nature is also one of the drawbacks of our approach. The accuracy results are still in the range below 80, which is a clear indication that these models fall short due to having smaller number of parameters compared to state of the art GPT, Gemini and LLAMA models (inferred from the “Densifying law of LLMs” [21]). So, as a future extension in requirement noise reduction, the viability of the state of the art models as well as stronger classification layers (stacking multiple classifiers for starters) remain as an yet to explore prospect.

7 Conclusion

In conclusion, this study contributed valuable insights into the application of transformer models in natural language processing tasks, specifically in the context of software requirements classification. By focusing on noise reduction, this research lays the foundation for enhancing the efficiency and effectiveness of the software development process. Our developed methodology showcased improved performance when compared with various out-of-the-box classifiers in terms of accuracy.

While the current work primarily addressed noise reduction in requirements, the future direction of this research involves extending the classification pipeline to encompass all research objectives, including improving requirement assessment, conflict resolution and multimodal analysis (involving charts, diagrams, wireframes - etc.).

References

- [1] Sadeen Alharbi. 2022. Ambiguity Detection in Requirements Classification Task using Fine-Tuned Transformation Technique. In *CS & IT Conference Proceedings*, Vol. 12. CS & IT Conference Proceedings.
- [2] Leo Breiman. 2001. Random forests. *Machine learning* 45 (2001), 5–32.
- [3] Corinna Cortes and Vladimir Vapnik. 1995. Support-vector networks. *Machine learning* 20 (1995), 273–297.
- [4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [5] Harris Drucker, Donghui Wu, and Vladimir N Vapnik. 1999. Support vector machines for spam categorization. *IEEE Transactions on Neural networks* 10, 5 (1999), 1048–1054.
- [6] Alessio Ferrari, Giorgio Oronzo Spagnolo, and Stefania Gnesi. 2017. Pure: A dataset of public requirements documents. In *2017 IEEE 25th International Requirements Engineering Conference (RE)*. IEEE, 502–505.
- [7] Vladimir Ivanov, Andrey Sadovykh, Alexandr Naumchev, Alessandra Bagnato, and Kirill Yakovlev. 2021. Extracting software requirements from unstructured documents. In *International Conference on Analysis of Images, Social Networks and Texts*. Springer, 17–29.
- [8] Thorsten Joachims. 1998. Text categorization with support vector machines: Learning with many relevant features. In *European conference on machine learning*. Springer, 137–142.
- [9] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).
- [10] Garima Malik, Mucahit Cevik, Devang Parikh, and Ayse Basar. 2022. Identifying the requirement conflicts in SRS documents using transformer-based sentence embeddings. *arXiv preprint arXiv:2206.13690* (2022).
- [11] Idza Aisara Norabid and Fariza Fauzi. 2022. Rule-based text extraction for multimodal Knowledge Graph. *International Journal of Advanced Computer Science and Applications* 13, 5 (2022).
- [12] Bashar Nuseibeh and Steve Easterbrook. 2000. Requirements engineering: a roadmap. In *Proceedings of the Conference on the Future of Software Engineering*. 35–46.
- [13] Klaus Pohl. 2016. *Requirements engineering fundamentals: a study guide for the certified professional for requirements engineering exam-foundation level-IREB compliant*. Rocky Nook, Inc.
- [14] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108* (2019).
- [15] Justyna Sarzynska-Wawer, Aleksander Wawer, Aleksandra Pawlak, Julia Szymanowska, Izabela Stefaniak, Michal Jarkiewicz, and Lukasz Okruszek. 2021. Detecting formal thought disorder by deep contextualized word representations. *Psychiatry Research* 304 (2021), 114135.
- [16] Daniel Siahaan and Brian Rizqi Paradisiaca Darnoto. 2022. A Novel Framework to Detect Irrelevant Software Requirements Based on MultiPhiLDA as the Topic Model. In *Informatics*, Vol. 9. MDPI, 87.
- [17] Marina Sokolova and Guy Lapalme. 2009. A systematic analysis of performance measures for classification tasks. *Information processing & management* 45, 4 (2009), 427–437.
- [18] Kentaro Torisawa et al. 2007. A new perceptron algorithm for sequence labeling with non-local features. In *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*. 315–324.
- [19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [20] Karl E Wiegiers and Joy Beatty. 2013. *Software requirements*. Pearson Education.
- [21] Chaojun Xiao, Jie Cai, Weilin Zhao, Biyuan Lin, Guoyang Zeng, Jie Zhou, Zhi Zheng, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025. Densifying law of llms. *Nature Machine Intelligence* (2025), 1–11.